EMNIST 데이터셋을 이용한 AE 기반 이상탐지 모델 최적화 및 분석

20202982 위찬우
20201599 이건아

2024.11.25

목 차

1
연구 배경

2
이론 & 연구과정

3
연구 결과 & 결론

연구 배경

연구 배경

- 산업 분야에서 이상탐지는 중요한 역할을 함
- 이상탐지에서 오토인코더는 정상 데이터만을 학습하고 비정상 데이터를 입력 시 큰 복원오차를 통해 이상 여부를 판별
- EMNIST 데이터셋을 이용하여 오토인코더 기반 이상탐지 모델을 최적화 하고 다양한 하이퍼파라미터와 모델의 구성 요소를 조정하여 성능을 극대화 함

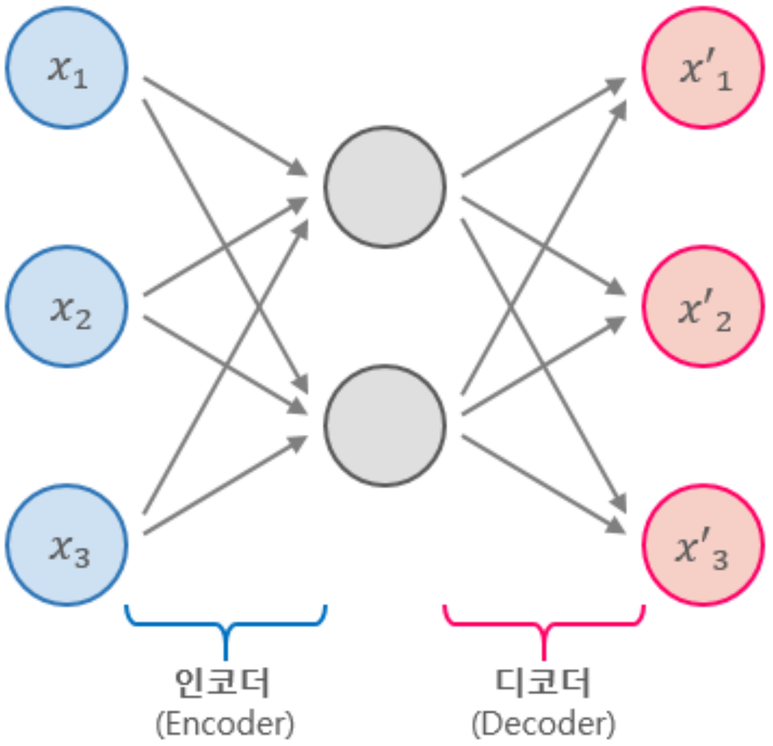


Fig. 1. Autoencoder 구조

이론 & 연구과정

데이터셋 구성

- 학습 데이터셋은 숫자 이미지로만 구성
- Validation & Test 데이터셋은 숫자와 영문자 이미지를 모두 포함하여 혼합된 형태로 구성
- 숫자는 정상 데이터, 문자는 이상 데이터로 간주하여 오토인코더가 영문자를 비정상 패턴으로 감지하도록 유도함



Fig. 2. EMNIST 데이터셋

이론 & 연구과정

Loss Function

MSE

예측 값과 실제 값 차이를 제곱하여 평균을 구하는 방식

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Huber Loss

MSE와 MAE의 특성을 결합하여 delta 매개변수를 통해 오차에 따라 페널티 부여

$$L_\delta(e) = \begin{cases} \frac{1}{2}a^2 & for|a| \le \delta, \\ \delta\left(|a| - \frac{1}{2}\delta\right), & otherwise \end{cases}$$

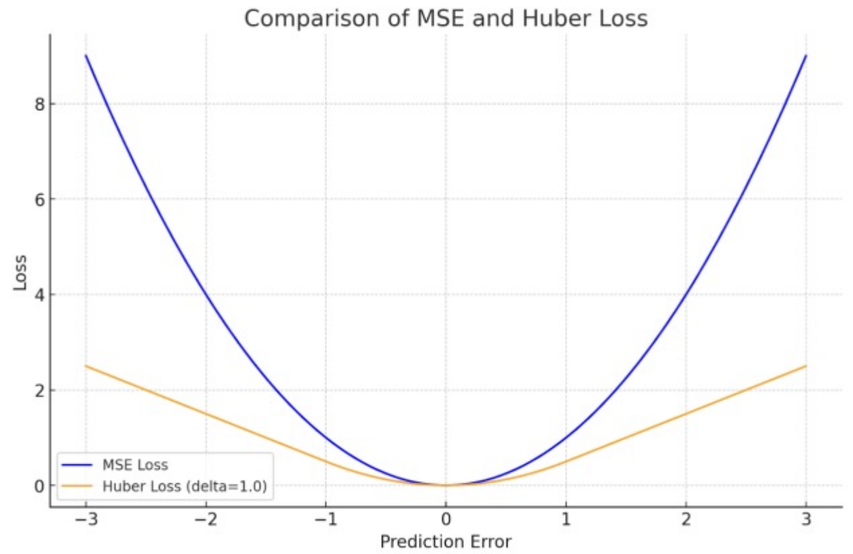


Fig. 3. Comparison of MSE and Huber Loss

이론 & 연구과정

Activation Function

ReLU

입력이 양수일땐 값을 출력하고, 음수일때 0을 출력하는 함수

$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

Leaky ReLU

ReLU에서 음수 입력에 대해 작은 기울기를 반환하여 학습에 포함시키는 함수

$$f(x) = \begin{cases} x & if\ x > 0, \\ 0.01x & otherwise \end{cases}$$

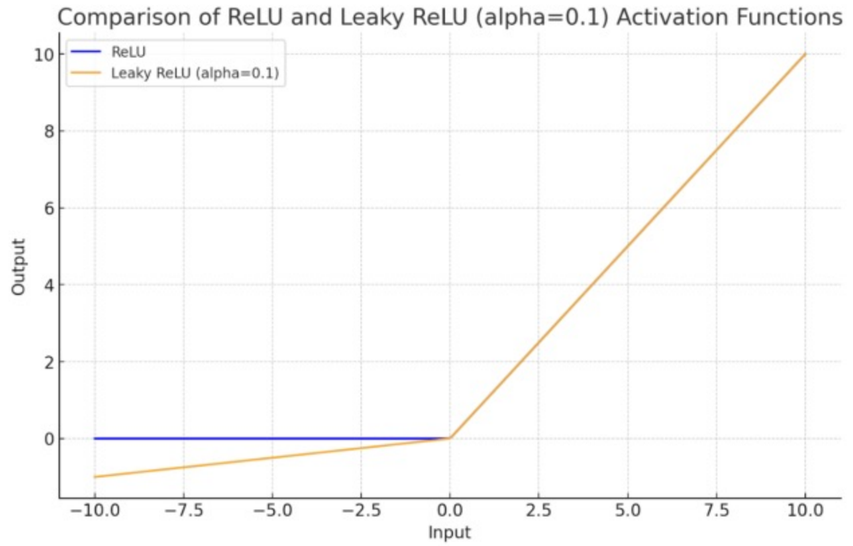


Fig. 4. Comparison of ReLU and Leaky ReLU

이론 & 연구과정

Optimization

Adam

학습률을 변수마다 다르게 조정하여 빠르게 최적화 수행

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 g_{t-1} + (1 - \beta_2) {g_t}^2$$
$$\widehat{m_t} = \frac{m_t}{1 - {\beta_1}^t}, \widehat{v_t} = \frac{v_t}{1 - {\beta_2}^t}$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\widehat{v_t}} + \epsilon} \widehat{m_t}$$

NAdam

기울기를 미리 반영하여 더 빠르게 수렴하도록 함

$$g_t = \nabla_{\theta_t} J(\theta_t)$$
$$m_t = \gamma m_{t-1} + \eta g_t$$
$$\theta_{t+1} = \theta_t - m_t$$
$$\therefore \theta_{t+1} = \theta_t - (\gamma m_{t-1} + \eta g_t)$$

이론 & 연구과정

하이퍼파라미터

- 1차 성능평가에서 validation set의 성능을 극대화 하는 과정에서 얻은 정보를 바탕으로 실제 모델 사용시 중요하게 여겨지는 test set에서의 이상치 탐지의 성능을 올리는 방향으로 하이퍼파라미터 조정
- 성능 지표들의 전반적은 밸런스 고려 및 F1-score를 주 성능 지표로 하이퍼파라미터 조정

하이퍼파라미터 튜닝 과정

batch_size	num_epochs	learning_rate	fisrt_hl	second_hl	third_hl	forth_hl	fifth_hl	sixth_hl	latent_size=32	act_fn	loss_fn	opt	Loss	valid_threshold
64	100	5.00E-04	550	380	210	80			32	LeakyReLU	HuberLoss(delta = 1.0)	NAdam	0.0033	0.05
64	100	5.00E-04	600	450	250	120	60		32	LeakyReLU	HuberLoss(delta = 1.0)	NAdam	0.0053	0.067
64	100	5.00E-04	550	400	200	120	70		32	LeakyReLU	HuberLoss(delta = 1.0)	NAdam	0.0047	0.0652
32	100	5.00E-04	550	400	200	120	70		32	LeakyReLU	HuberLoss(delta = 1.0)	NAdam	0.0044	0.0601
64	300	1.00E-04	550	400	200	120	70		32	LeakyReLU	HuberLoss(delta = 1.0)	NAdam	0.0051	0.0671
64	100	7.00E-04	600	400	200	120	80		32	LeakyReLU	HuberLoss(delta = 1.0)	NAdam	0.0045	0.0594
32	200	7.00E-04	600	450	250	150	80		32	LeakyReLU	HuberLoss(delta = 1.0)	NAdam	0.0032	0.0513
32	100	7.00E-04	600	450	250	150	80		32	LeakyReLU	HuberLoss(delta = 1.0)	NAdam	0.0043	0.0594
64	100	5.00E-04	600	450	250	150	80		32	LeakyReLU	HuberLoss(delta = 1.0)	NAdam	0.0045	0.0622
64	100	5.00E-04	600	450	250	150	80		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0058	0.0716
64	100	5.00E-04	600	450	250	150	80		32	PReLU	HuberLoss(delta = 1.0)	NAdam	0.0031	0.0504
64	100	5.00E-04	600	450	250	150	80		32	ELU	HuberLoss(delta = 1.0)	NAdam	0.0026	0.0424
32	200	5.00E-04	600	450	250	150	80		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0042	0.0618
32	100	5.00E-04	620	470	280	150	70		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0052	0.0694
64	100	5.00E-04	620	470	280	150	70		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0061	0.0756
64	100	5.00E-04	580	400	250	120	70		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0061	0.0731
64	100	7.00E-04	600	400	230	110	60		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0059	0.0732
128	100	5.00E-04	600	400	230	110	60		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0079	0.0889
128	300	1.00E-04	600	400	220	110	60		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0074	0.0802
64	100	5.00E-04	600	400	200	100	60		32	ReLU	HuberLoss(delta = 1.0)	Adam	0.0063	0.0758
64	100	5.00E-04	600	400	200	100			32	ReLU	HuberLoss(delta = 1.0)	Adam	0.0035	0.053
64	100	5.00E-04	550	400	250	150			32	ReLU	HuberLoss(delta = 1.0)	Adam	0.004	0.0562
64	100	5.00E-04	550	350	200	80			32	ReLU	HuberLoss(delta = 1.0)	Adam	0.0041	0.0569
64	100	5.00E-04	600	450	250	150	80		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0058	0.0716
64	100	5.00E-04	600	450	250	150	80		32	ReLU	HuberLoss(delta = 1.0)	Adam	0.0058	0.0701
64	100	5.00E-04	600	450	250	150	80		32	ReLU	MSELoss()	NAdam	0.0115	0.0709
64	100	5.00E-04	580	400	240	120	60		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0059	0.0685
64	100	5.00E-04	580	400	240	120	60		32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0059	0.0685
32	100	5.00E-04	620	480	280	150	90	60	32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0069	0.0814
64	100	5.00E-04	620	480	280	150	90	60	32	ReLU	HuberLoss(delta = 1.0)	NAdam	0.0077	0.0802

Table 1. 하이퍼파라미터 튜닝 과정

이론 & 연구과정

Layer 수

fisrt_hl	*second_hl*	*third_hl*	forth_hl	fifth_hl	*valid_F1-Score*	*test_F1-Score*
600	450	250			0.896	0.7684
600	450	250	150		0.8991	0.8
600	450	250	150	80	0.8757	0.8127

Table 2. hidden layer 수에 따른 F1-score

Layer 차원 수

fisrt_hl	*second_hl*	*third_hl*	forth_hl	fifth_hl	*valid_F1-Score*	*test_F1-Score*
580	400	240	120	60	0.8765	0.8023
600	450	250	150	80	0.8757	0.8127
620	470	280	150	70	0.8771	0.8061

Table 3. layer 차원 수에 따른 F1-score

Epoch

num_epochs	*valid_F1-Score*	*test_F1-Score*
50	0.8344	0.797
100	0.8757	0.8127
150	0.8981	0.8027

Table 4. Epoch 따른 F1-score

Batch Size

batch_size	*valid_F1-Score*	*test_F1-Score*
32	0.8872	0.8053
64	0.8757	0.8127
128	0.8694	0.8109

Table 5. Batch size 따른 F1-score

Learning Rate

learning_rate	*valid_F1-Score*	*test_F1-Score*
1.00E-04	0.8158	0.7819
5.00E-04	0.8757	0.8127
1.00E-03	0.8883	0.8058

Table 6. Learning Rate에 따른 F1-score

연구결과

최종 선정 모델 및 성능

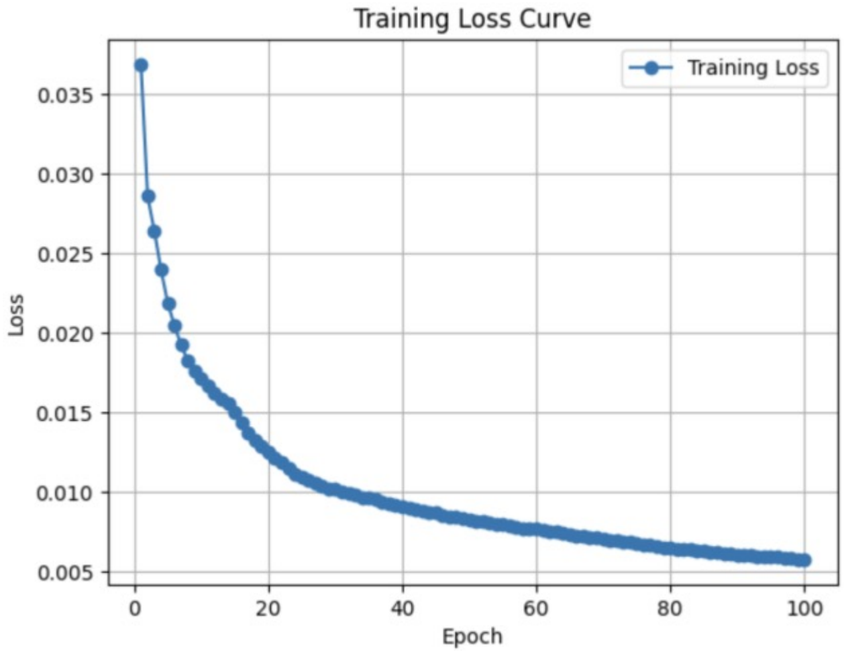
batch_size	num_epochs	learning_rate	fisrt_hl	second_hl	third_hl	forth_hl	fifth_hl
64	100	5.00E-04	600	450	250	150	80

latent_size=32	act_fn	loss_fn		opt
32	ReLU	HuberLoss(delta = 1.0)		NAdam

Loss	valid_threshold	valid_Accuracy	valid_Precision	valid_Recall	valid_F1-Score
0.0058	0.0716	0.8825	0.9292	0.8281	0.8757

test_Accuracy	test_Precision	test_Recall	test_F1-Score
0.8097	0.8001	0.8257	0.8127

Traning Loss Curve



결론

제약이 있어 실제 산업계에서 사용하기엔 개선이 필요한 모델

특정 도메인의 요구사항과 데이터 특성에 맞춰 다양한 구조와 최적화 기법을 사용해 성능을 극대화 할 수 있을 것으로 보임

Reference

[1] Sangchul Yoo, “Performance Comparison of Autoencoder Anomaly Detection Algorithms.” Master’s thesis, Ajou University, 2023, Gyeonggi-do, South Korea.
(https://www.riss.kr/search/detail/DetailView.do?p_mat_type=be54d9b8bc7cdb09&control_no=3ccaaba61dd227ecffe0bdc3ef48d419&keyword=%EC%98%A4%ED%86%A0%EC%9D%B8%EC%BD%94%EB%8D%94)
[2] Miru Kim, Hyoseon Kye, Minhae Kwon,Network Anomaly Detection System Using Hidden Layer Information of Autoencoder, *The Journal of Korean Institute of Communications and Information Sciences '22-09 Vol.47 No.09*(https://doi.org/10.7840/kics.2022.47.9.1310)
[3] S. K. Dana, “Incremental Learning of Handwritten Characters in EMNIST dataset,” *2023 International Conference for Advancement in Technology* (ICONAT) (https://ieeexplore.ieee.org/document/10080724)

https://comdon-ai.tistory.com/49 : 함수 식 참고
https://aws.amazon.com/ko/what-is/hyperparameter-tuning/ : 하이퍼파라미터 튜닝이란?
https://taehojo.github.io/deeplearning-for-everyone/auto-encoder.html : Fig. 1. 오토인코더 구조

THANK YOU